

K-Pick Website App

Operations Manual

Complete handoff documentation for running, troubleshooting, and maintaining the website, quote/PO workflow, staff dashboard, Railway deployment, SQLite database, inventory sync, and PDF generation.

Generated: 2026-05-10

Project folder: C:\WebDraft

Primary runtime: Node.js 24+

K-Pick Website App Operations Manual

Generated: 2026-05-10

Project folder: C:\WebDraft

Application: K-Pick Trading Corp website, quote/PO builder, staff dashboard, and quote PDF backend

1. Emergency Quick Start

If the website or PO system is down, do these checks in order.

- * Open the health check:
 - Local: `http://127.0.0.1:8000/api/health`
 - Railway: open the deployed site URL and add `/api/health`
- * If health returns JSON with "ok": true, the backend is running. Try:
 - Customer quote page: `/request.htm`
 - Staff dashboard: `/request-admin.htm`
- * If the health check does not load, check the host:
 - Railway: open the service logs and confirm npm start is running.
 - Local PC: run `npm run dev` from C:\WebDraft.
- * If the staff dashboard opens but login fails, confirm the staff account exists and is active. Admin can reset staff passwords in `/request-admin.htm`.
- * If products show old prices or stock, log in as Admin and press Sync Sheet. If that fails, check the Google Sheet CSV URL and backend logs.
- * If generated PDFs fail, confirm the PO exists, the PDF link includes a token, or the staff user is logged in.
- * If data appears missing after deployment, check `KPICK_DATA_DIR`. The SQLite database must live on a persistent Railway volume. Without a persistent volume, new deployments can start with an empty database.

2. What This App Does

This repository contains a public website and an internal operations backend.

Public pages:

- * `index.html`: homepage and brand overview.
- * `sungshim.html`, `hansol.html`, `erop.html`: brand landing pages.
- * Product detail pages such as `sungshim-insulin-syringe.html` and `erop-trocar.html`.
- * `contact.htm`: contact form using Web3Forms.
- * `request.htm`: customer quote / PO request builder.

Staff page:

- * `request-admin.htm`: login dashboard for Admin, CS, Inventory, Manager, and Boss roles.

Backend:

- * `backend/server.mjs`: Node server that serves static files, products, quote requests, staff login, admin actions, reports, Google Sheet inventory sync, and quote PDFs.
- * `backend/server.py`: fallback Python server kept in the repo, but the active deployment uses Node.

Deployment:

- * `railway.json`: Railway config. It runs `npm install`, starts with `npm start`, and checks `/api/health`.

3. Important Files

- * package.json: Node scripts and dependency list.
- * .nvmrc: required Node major version, currently 24.
- * railway.json: Railway build/start/health settings.
- * backend/server.mjs: main app server and API.
- * backend/seed_products.json: starting product catalog inserted into SQLite.
- * backend/google-sheet-inventory-template.csv: expected Google Sheet column format.
- * css/style.css: all page styling.
- * script/request-po.js: customer quote builder logic.
- * script/request-admin.js: staff dashboard logic.
- * script/request-products.js: local fallback product list if API is unavailable.
- * backend/data/kpick_quote.sqlite3: local SQLite database when KPICK_DATA_DIR is not set. This folder is ignored by Git.
- * backend/data/server.log: local backend log when KPICK_DATA_DIR is not set.

4. Runtime Requirements

Required:

- * Node.js 24 or newer.
- * npm.
- * Writable data folder for SQLite.

The backend uses Node's built-in node:sqlite, so Node 24+ is required. Older Node versions can fail immediately on startup.

Install dependencies:

```
npm install
```

Run locally:

```
npm run dev
```

Open locally:

```
http://127.0.0.1:8000/request.htm  
http://127.0.0.1:8000/request-admin.htm
```

Production start command:

```
npm start
```

5. Environment Variables

Set these in Railway variables or before starting locally.

Required in hosted production:

- * KPICK_ADMIN_PASSWORD: fallback admin password and initial default account password.
- * KPICK_AUTH_SECRET: long random secret used to sign staff tokens and PDF links.

Strongly recommended:

- * KPICK_DATA_DIR: persistent directory for SQLite and logs. On Railway, set this to the mounted volume path, commonly /data.
- * KPICK_AUTO_SYNC_INVENTORY: 1 to sync on startup and schedule, 0 to disable.
- * KPICK_INVENTORY_SYNC_INTERVAL_MS: default 1800000 or 30 minutes.
- * KPICK_INVENTORY_SYNC_TIMEOUT_MS: default 15000.
- * KPICK_INVENTORY_CSV_URL: Google Sheet CSV export URL. If not set, the app uses the URL hardcoded in backend/server.mjs.

- * KPICK_HOST: optional. The app binds to 0.0.0.0 automatically when Railway provides PORT.
- * KPICK_PORT: optional local override. Default is 8000; Railway uses PORT.

Generate a strong auth secret:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

Do not commit real secrets to Git.

6. Railway Deployment Notes

Railway reads railway.json.

Build command:

```
npm install
```

Start command:

```
npm start
```

Health check:

```
/api/health
```

Before production use, confirm:

- * A persistent Railway volume is attached.
- * KPICK_DATA_DIR points to that volume.
- * KPICK_ADMIN_PASSWORD is set.
- * KPICK_AUTH_SECRET is set.
- * The service logs show K-Pick backend running.
- * /api/health returns JSON.

If Railway redeploys and all POs disappear, the most likely cause is that SQLite was stored in a temporary filesystem instead of a volume.

7. Data Storage

SQLite tables created by backend/server.mjs:

- * products: product catalog, stock, price, carton discount data, active flag, sync timestamp.
- * quote_requests: customer PO requests, items JSON, totals JSON, workflow fields, deleted/cancelled metadata.
- * staff_users: staff accounts, password hashes, manager code hashes, role, active flag.
- * quote_audit_log: important staff actions and changes.

Default local database path:

```
backend/data/kpick_quote.sqlite3
```

Production database path:

```
KPICK_DATA_DIR/kpick_quote.sqlite3
```

Backup the SQLite file regularly. To restore, stop the service, replace the SQLite file with a known good backup, then restart.

8. Default Staff Accounts

On first startup, the app creates default local accounts:

- * admin / default password / Admin
- * cs001 / default password / CS
- * inv001 / default password / Inventory

- * mgr001 / default password / Manager
- * boss001 / default password / Boss

The default password comes from KPICK_ADMIN_PASSWORD; locally it falls back to kpick0324admin.

After first deployment:

- * Log in as admin.
- * Create named staff accounts.
- * Reset default passwords.
- * Keep at least one active Admin account.

Admin can add, edit, rename, reset passwords/codes, and deactivate staff accounts from /request-admin.htm.

9. Staff Roles And Permissions

Admin:

- * Can view and modify all POs.
- * Can sync Google Sheet inventory.
- * Can manage staff accounts.
- * Can edit products, totals, workflow, customer details, and staff assignment.
- * Can delete/cancel eligible POs.

CS:

- * Can update customer details.
- * Can add SI number.
- * Must assign inventory staff after entering SI number.
- * Can change products, quantities, prices, or totals only before payment/generated workflow and only with a Manager/Admin manager code.
- * Can delete/cancel unpaid generated POs when customer cancelled or is unreachable.

Inventory:

- * Can update PO condition only for assigned POs.
- * Allowed conditions: Picking Products, For Repacking, Ready for Shipment, Released / Shipped.
- * Can mark ready for shipment when assigned.

Manager:

- * Can view work queue and reports.
- * Manager code authorizes CS order, price, and total revisions.

Boss:

- * Can view work queue and reports.

10. Normal PO Workflow

- * Customer or staff opens /request.htm.
- * They enter customer details.
- * They select products and quantities.
- * The app calculates subtotal, carton discount, and grand total.
- * They press Submit Request.
- * Backend saves the request and returns a generated PO number and PDF link.
- * Staff opens /request-admin.htm.
- * CS logs in and reviews new generated POs.
- * CS enters SI number and assigns Inventory staff.
- * PO moves to Pending for Picking.
- * Assigned Inventory staff updates condition through:
 - Picking Products

- For Repacking
- Ready for Shipment
- Released / Shipped
- * Manager/Boss/Admin can review reports.

Workflow statuses:

- * Generated
- * Pending for Picking
- * Picking Products
- * For Repacking
- * Ready for Shipment
- * Released / Shipped

11. Product And Inventory Sync

Seed products are in:

```
backend/seed_products.json
```

Google Sheet sync expects columns like:

```
sku, brand, category, product_name, packaging, carton_packing, gauge, stock_quantity, stock_unit, unit_price, active, last_updated, notes
```

The current backend sync updates existing SKUs. It does not create brand-new unknown SKUs from the Sheet.

If a new SKU is needed:

- * Add it to backend/seed_products.json.
- * Deploy/restart so the seed runs.
- * Add matching SKU row in the Google Sheet.
- * Use Admin Sync Sheet.

The sync mainly updates:

- * Product name.
- * Category.
- * Stock quantity.
- * Stock unit.
- * Unit price.
- * Last synced timestamp.

Auto sync:

- * On server startup, if KPICK_AUTO_SYNC_INVENTORY is not 0.
- * Every 30 minutes by default.

Manual sync:

- * Log in as Admin at /request-admin.htm.
- * Press Sync Sheet.

12. Quote PDF Behavior

PDFs are generated by backend/server.mjs using pdfkit.

PDF endpoint:

```
GET /api/quote-requests/:id/pdf
```

Access is allowed when:

- * The PDF URL includes a valid signed token, or
- * A logged-in staff user with a valid role requests it.

The PDF includes:

- * K-Pick logo when `img/KpickLogoDark.png` exists.
- * PO number.
- * Created date.
- * Customer details.
- * Selected products.
- * Unit price, quantity, discount, line total.
- * Subtotal, carton discount, grand total.

If a PDF URL stops working, generate a new PO modal link from the app or open it while logged in.

13. API Reference

Public:

- * GET `/api/health`: server/database health.
- * GET `/api/products`: active product catalog.
- * POST `/api/quote-requests`: create a quote/PO request.
- * GET `/api/quote-requests/:id/pdf?token=...`: signed PDF download.

Staff:

- * POST `/api/login`: staff login.
- * GET `/api/quote-requests`: list saved POs for logged-in roles.
- * PATCH `/api/quote-requests/:id`: update customer details, order revisions, workflow, or assignment according to role permissions.
- * DELETE `/api/quote-requests/:id`: cancel/hide eligible POs for Admin/CS.
- * POST `/api/inventory/sync`: Admin Google Sheet sync.
- * GET `/api/reports/summary?month=YYYY-MM`: Manager/Boss/Admin reports.
- * GET `/api/staff-users`: Admin staff list.
- * POST `/api/staff-users`: Admin create/update staff account.
- * DELETE `/api/staff-users/:id_number`: Admin deactivate staff account.
- * GET `/api/inventory-staff`: list active Inventory users for assignment.

14. Troubleshooting

Problem: App will not start.

- * Check Node version with `node -v`; must be 24+.
- * Run `npm install`.
- * Confirm `KPICK_ADMIN_PASSWORD` and `KPICK_AUTH_SECRET` are set when hosted.
- * Check Railway logs or `backend/data/server.log`.

Problem: `/api/health` works but pages 404.

- * Confirm static files exist in the project root.
- * Use exact filenames such as `/request.htm` and `/request-admin.htm`.
- * The backend intentionally blocks private paths like `/backend/` and `/node_modules/`.

Problem: Quote page says it is using local draft product list.

- * Product API failed or backend is not serving `/api/products`.
- * Check `/api/health`.
- * Confirm the page is opened through the Node backend, not just double-clicked as a local file.

Problem: Staff dashboard says API did not return JSON.

- * Open dashboard through backend URL, for example `http://127.0.0.1:8000/request-admin.htm`.
- * Do not open `request-admin.htm` directly from the filesystem.

Problem: Staff cannot log in.

- * Confirm ID number and password.
- * Admin can reset password in Staff Accounts.
- * Confirm the staff user is active.

Problem: CS cannot save an SI workflow update.

- * CS must assign Inventory staff before saving after SI number is entered.

Problem: CS cannot change product/price/quantity.

- * The PO may already be paid or beyond Generated.
- * CS changes to products, price, quantity, removals, or totals require a Manager/Admin manager code.

Problem: Inventory cannot update a PO.

- * Inventory can only update assigned POs.
- * Confirm CS assigned the correct Inventory ID/name.

Problem: Google Sheet sync fails.

- * Confirm the Sheet is published or available as CSV.
- * Confirm the URL in KPICK_INVENTORY_CSV_URL or backend/server.mjs.
- * Confirm the Sheet has a sku column.
- * Check logs for timeout or HTTP status.

Problem: New SKU in Google Sheet does not appear.

- * Add the SKU to backend/seed_products.json first, then restart/deploy and sync.

Problem: Data disappeared after deploy.

- * Confirm Railway has a persistent volume.
- * Confirm KPICK_DATA_DIR points to the mounted volume.
- * Restore from SQLite backup if necessary.

Problem: Contact form does not send.

- * contact.htm posts to Web3Forms.
- * Confirm the hidden access_key in contact.htm is valid.
- * If Web3Forms is down, customers can email kpickmedicalmarketing@gmail.com.

15. Safe Maintenance Checklist

Before editing:

- * Run git status --short.
- * Make a copy of the SQLite database if changing backend logic.
- * Confirm you know whether changes are for static pages, dashboard JS, or backend routes.

After editing:

- * Run local server.
- * Check /api/health.
- * Open /request.htm.
- * Create a test PO.
- * Download the generated PDF.
- * Log in to /request-admin.htm.
- * Update workflow as CS/Admin.
- * If inventory logic changed, test assignment and Inventory login.
- * If deployment changed, verify Railway logs and /api/health.

Before production deployment:

- * Make sure no secrets are committed.
- * Make sure .env and backend/data/ are not committed.
- * Confirm railway.json still uses npm start.

- * Confirm Node version remains compatible with node:sqlite.

16. Backup And Recovery

What to back up:

- * SQLite database file: kpick_quote.sqlite3.
- * Current code repository.
- * Railway environment variable values.
- * Google Sheet inventory source.

Suggested backup routine:

- * Download/copy the SQLite file at least daily while POs are actively generated.
- * Keep backups with date/time in the filename.
- * Keep one backup before every code deployment.

Restore routine:

- * Stop the running backend service.
- * Copy the known good SQLite file into KPICK_DATA_DIR.
- * Start the service.
- * Open /api/health.
- * Log in to /request-admin.htm and confirm recent POs appear.

17. Developer Notes

Frontend pages are plain HTML, CSS, and JavaScript. There is no build step for the frontend.

Backend is a single Node module:

```
backend/server.mjs
```

The server:

- * Creates/migrates tables at startup.
- * Seeds default staff accounts.
- * Seeds products from backend/seed_products.json.
- * Optionally syncs inventory from Google Sheet.
- * Serves public static files.
- * Blocks backend and dependency files from public static access.

Current npm scripts:

```
{
  "dev": "node --watch backend/server.mjs",
  "start": "node backend/server.mjs"
}
```

18. Handoff Summary

If the owner is unavailable, the app can still be managed by someone who has:

- * Railway access.
- * Repository access.
- * Staff Admin login.
- * Google Sheet access.
- * SQLite backup access.

The most critical operational facts are:

- * Use Node 24+.
- * Keep KPICK_DATA_DIR on persistent storage.
- * Keep KPICK_AUTH_SECRET stable; changing it invalidates existing sessions and signed PDF links.

- * Back up SQLite.
- * Manage staff users through Admin dashboard.
- * Add new SKUs to seed data before expecting Google Sheet sync to update them.

